

Lab 8: RNN, LSTM and Attention

1. Introduction

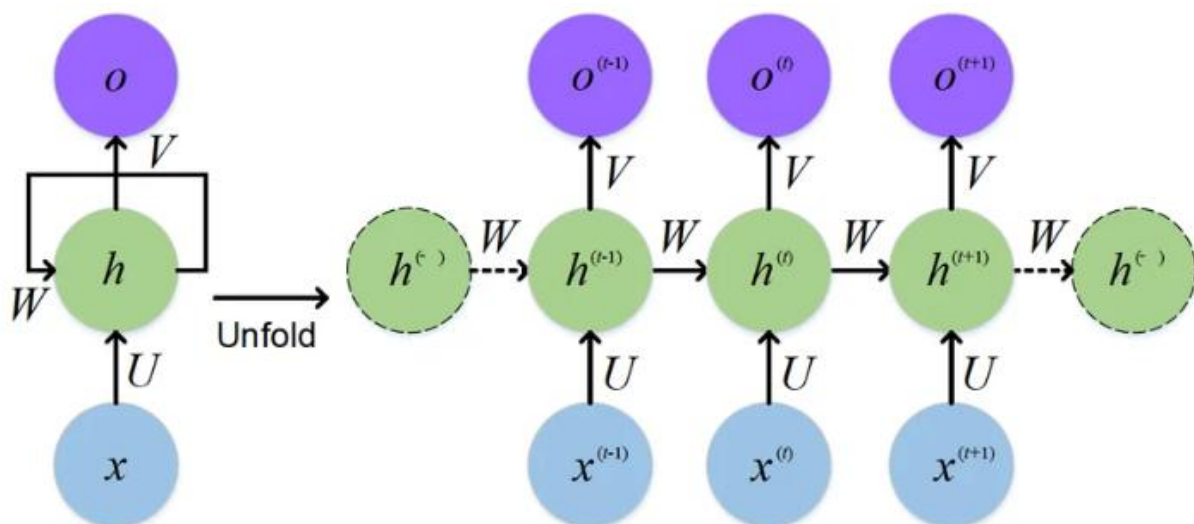
1.1 Sequence data and the need for memory

Many types of data are **sequences**, in which the order of the items carries meaning. Examples include the words in a sentence, the notes in a piece of music, daily stock prices and sensor readings over time. Each item in a sequence is called a **step**.

A model that processes a sequence often needs information from an earlier step to make a decision at a later step. Consider the sentence "I grew up in France, and after many years of study I now speak fluent ...". The word "France" appears many steps before the word that it helps to predict. A model that does not retain this information cannot complete the sentence correctly. This lab studies three model families that address this requirement in different ways: the recurrent neural network (RNN), the long short-term memory network (LSTM) with its variant the gated recurrent unit (GRU), and attention.

1.2 Recurrent neural network (RNN)

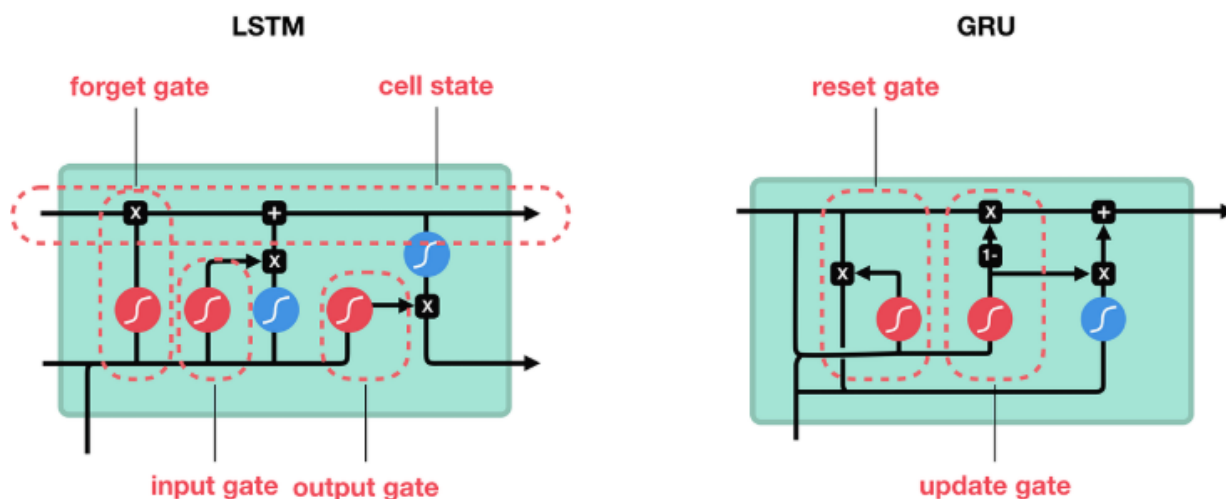
Purpose. An RNN processes a sequence one step at a time. At each step, it reads the current input and updates a vector called the **hidden state**. The hidden state is the memory of the model: it summarises what the network has seen so far. Because the same computation is repeated at every step, an RNN can process sequences of any length.



Limitation. An RNN has difficulty retaining information over many steps. The hidden state is overwritten at every step, so old information is gradually lost. Training is also difficult. A model is trained using the **gradient**, a signal that indicates how each weight should change to reduce the error. To learn from an early step, this signal must be passed backwards through every later step, and it usually becomes smaller at each step. For a long sequence it is close to zero, so the model receives almost no signal about the early steps. This is known as the **vanishing gradient** problem.

1.3 LSTM and GRU

Purpose. The LSTM was proposed to improve the memory of the RNN. It adds a separate memory, called the cell state, and **gates**. A gate is a learned switch that decides what information to write, keep or discard at each step. The GRU is a simplified variant of the LSTM with fewer gates.



Limitation. Gates make it easier to learn long-range relationships, but they do not remove the problem. Information is still passed from one step to the next, so very long sequences remain difficult. The steps must also be processed in order, which limits computation speed.

1.4 Attention

Purpose. Attention allows a model to relate any two steps directly, without passing information through all the steps between them. For each step, the model computes a weight for every other step, and then combines the information from those steps according to the weights. All steps can be processed in parallel. Attention is the core component of the Transformer architecture, which is the basis of modern large language models (LLMs).

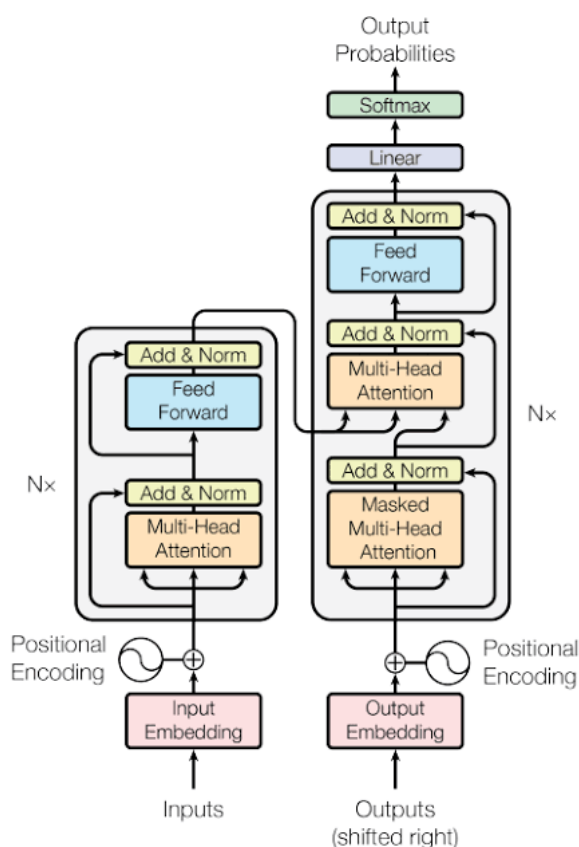


Figure 1: The Transformer - model architecture.

Requirement and limitation. Attention does not use the order of the steps by itself. **Positional encoding** solves this by adding a fixed pattern to each step, so that the model can identify the position of that step. Attention also compares every pair of steps, so the amount of computation grows with the square of the sequence length.

Table 1. Summary of the three model families.

Model	Purpose	Main limitation
RNN	Process a sequence step by step, using a hidden state as memory.	Early information fades. Training on long sequences is difficult (vanishing gradient).
LSTM, GRU	Improve the memory of the RNN with gates that control what is kept or discarded.	Better than the RNN, but information is still passed step by step, so long sequences remain difficult.
Attention	Relate any two steps directly. Process all steps in parallel.	Needs positional encoding. Computation grows with the square of the sequence length.

1.5 The task used in this lab

This lab uses a simple task to test how far back each model can remember. Each example is a sequence of T random digits, for example 4 9 3 0 3 9 7 3 7 3 ($T = 10$). The label is the **first** digit of the sequence, which is 4 in this example. The other digits contain no useful information. A model can therefore give the correct answer only if it retains the information from step 1 until the last step.

This task is chosen because it isolates memory. Real text or stock data contain many other patterns, so it would be unclear whether a model succeeded because it remembered or for some other reason. Here, accuracy measures memory directly. A model that guesses achieves an accuracy of 0.10, because there are ten possible digits.

The sequence length T is the variable that we change. When T is small, the model needs to remember for a short time. As T increases, the model must retain the information for longer. The three parts of the lab follow from this:

- **Part 1** measures how the gradient decreases with the number of steps. This is the cause of the difficulty described in Section 1.2.
- **Part 2** trains an RNN, an LSTM and a GRU for increasing values of T , and records where each model stops working.
- **Part 3** applies attention to the same task, examines which step the model looks at, and shows why positional encoding is necessary.

2. Objectives

After completing this lab, you will be able to:

- measure the vanishing gradient in an RNN, an LSTM and a GRU;
- compare the accuracy of the three models as the sequence length increases;
- apply self-attention to the same task and interpret its attention weights;
- explain the role of positional encoding.

Tools needed: Python 3.10 or later, Jupyter, and `pip install torch matplotlib numpy`. No GPU is required.

3. Setup

Run this cell.

```
import numpy as np, torch, torch.nn as nn
import matplotlib.pyplot as plt

def make_data(n, T, seed):
    g = torch.Generator().manual_seed(seed)
    X = torch.randint(0, 10, (n, T), generator=g)  # n sequences of T digits
    return X, X[:, 0].clone()                    # label = the FIRST digit

X, y = make_data(3, 10, seed=0)
print(X)
```

```
print(y)
```

You should see 3 rows of 10 digits and 3 labels. Check that each label is the first digit of its row.

4. Part 1: Vanishing gradients

Here, the gradient at step t means: how much the last hidden state changes if you nudge the input at step t . If the gradient at step 0 is tiny, the model gets no signal from step 0.

The cell below feeds a random sequence of 50 steps into each layer and measures the gradient at every step. Run it.

```
def grad_per_step(layer, T=50):
    x = torch.randn(64, T, 16, requires_grad=True) # random input sequence
    h, _ = layer(x) # h: hidden state at every step
    g = torch.autograd.grad(h[:, -1].pow(2).sum(), x)
    return g.norm(dim=-1).mean(0).detach() # gradient size at each input step

plt.figure(figsize=(6, 3.5))
for name, Layer in [("RNN", nn.RNN), ("LSTM", nn.LSTM), ("GRU", nn.GRU)]:
    torch.manual_seed(0)
    n = grad_per_step(Layer(16, 32, batch_first=True))
    plt.semilogy(n.numpy(), label=name)
    print(name, "gradient at last step / first step = %.1e" % (n[-1] / n[0]))
plt.xlabel("input step (0 = first)"); plt.ylabel("gradient size")
plt.legend(); plt.tight_layout(); plt.show()
```

Typical result: the ratio (last step / first step) is about **1.5e+13** for RNN, **7.8e+10** for LSTM and **2.8e+10** for GRU. All three shrink. RNN shrinks by far the most.

5. Part 2: RNN, LSTM and GRU

The three models are the same except for the recurrent layer. Each reads the whole sequence and predicts the first digit from the last hidden state. `train_eval` trains for 15 epochs and returns test accuracy. Chance is 0.10.

```
class RNNNet(nn.Module):
    def __init__(self, kind, units=32):
        super().__init__()
        self.emb = nn.Embedding(10, 16)
        Layer = {"rnn": nn.RNN, "lstm": nn.LSTM, "gru": nn.GRU}[kind]
        self.rnn = Layer(16, units, batch_first=True)
        self.out = nn.Linear(units, 10)

    def forward(self, x):
        h, _ = self.rnn(self.emb(x))
        return self.out(h[:, -1]) # predict from the last step only

def train_eval(model, T, seed, epochs=15):
    Xtr, ytr = make_data(5000, T, seed)
    Xte, yte = make_data(1000, T, seed + 1000)
    opt = torch.optim.Adam(model.parameters(), lr=1e-3)
    loss_fn = nn.CrossEntropyLoss()
    for _ in range(epochs):
        perm = torch.randperm(len(Xtr))
        for i in range(0, len(Xtr), 64):
            idx = perm[i:i + 64]
            opt.zero_grad()
            loss_fn(model(Xtr[idx]), ytr[idx]).backward()
            opt.step()
        model.eval()
        with torch.no_grad():
            return (model(Xte).argmax(1) == yte).float().mean().item()

def run(name, make, Ts, seeds=(0, 1, 2), epochs=15):
    for T in Ts:
```

```
accs = []
for s in seeds:
    torch.manual_seed(s)
    accs.append(train_eval(make(T), T, s, epochs))
print(f"{name:10s} T={T:3d} mean={np.mean(accs):.2f} runs={np.round(accs, 2)}")
```

Now run 3 models x 4 lengths x 3 seeds. This takes about 5 minutes. Start it, then read Part 3 while it runs.

```
for kind in ["rnn", "lstm", "gru"]:
    run(kind, lambda T: RNNNet(kind), [5, 10, 20, 60])
```

Typical result (mean accuracy of 3 seeds; chance = 0.10):

Model	T = 5	T = 10	T = 20	T = 60
RNN	1.00	0.66	0.10	0.10
LSTM	1.00	0.96	0.47	0.09
GRU	1.00	1.00	0.10	0.09

Experiment: change one parameter

Make the hidden layer bigger: 32 units to 128 units. Run T = 20 again and compare with the table.

```
for kind in ["rnn", "lstm", "gru"]:
    run(kind + "-128", lambda T: RNNNet(kind, units=128), [20])
```

Typical result at T = 20: RNN 0.85, LSTM 0.59, GRU 1.00.

6. Part 3: Self-attention

Attention has no recurrence. Every position looks at every other position directly, so step 1 is one hop from step T.

But attention does not know the order by itself. Positional encoding fixes this: it adds a fixed sine and cosine pattern to each step, so every position gets its own tag. The cell below defines the model and runs it for T = 10, 20, 60.

```
def pos_enc(T, d):
    pos = torch.arange(T).unsqueeze(1).float()
    i = torch.arange(d).unsqueeze(0)
    angle = pos / torch.pow(10000.0, (2 * (i // 2)).float() / d)
    return torch.where(i % 2 == 0, torch.sin(angle), torch.cos(angle))

class AttnNet(nn.Module):
    def __init__(self, T, use_pos=True, d=32):
        super().__init__()
        self.use_pos = use_pos
        self.emb = nn.Embedding(10, d)
        self.register_buffer("pe", pos_enc(T, d))
        self.attn = nn.MultiheadAttention(d, 2, batch_first=True)
        self.ln1, self.ln2 = nn.LayerNorm(d), nn.LayerNorm(d)
        self.ff = nn.Sequential(nn.Linear(d, 64), nn.ReLU(), nn.Linear(64, d))
        self.out = nn.Linear(d, 10)

    def forward(self, x, return_w=False):
        h = self.emb(x)
        if self.use_pos:
            h = h + self.pe # add position information
        a, w = self.attn(h, h, h, need_weights=True, average_attn_weights=False)
        h = self.ln1(h + a)
        h = self.ln2(h + self.ff(h))
        o = self.out(h[:, -1]) # last position makes the prediction
        return (o, w) if return_w else o
```

```
run("attn", lambda T: AttnNet(T), [10, 20, 60])
```

Typical result: accuracy 1.00, 1.00, 1.00 for T = 10, 20, 60.

Where does the model look?

Train one model at T = 20. Then average the attention weights of the last position over 200 sequences and plot them for both heads.

```
T = 20
torch.manual_seed(0)
model = AttnNet(T)
print("accuracy:", train_eval(model, T, seed=0))

Xs, _ = make_data(200, T, seed=5)
model.eval()
with torch.no_grad():
    _, w = model(Xs, return_w=True)          # shape: (200, heads, T, T)
    last = w[:, :, -1, :].mean(0)           # last position, averaged over 200 sequences

fig, axes = plt.subplots(1, 2, figsize=(8, 2.8), sharey=True)
for h in range(2):
    axes[h].bar(range(T), last[h].numpy())
    axes[h].set_title(f"Head {h}"); axes[h].set_xlabel("position looked at")
axes[0].set_ylabel("attention weight")
plt.tight_layout(); plt.show()
```

Typical result: the last position puts most of its weight on position 0 (0.95 for head 0, 0.80 for head 1). The model looks straight at the first digit.

Ablation: remove positional encoding

```
run("attn_nopos", lambda T: AttnNet(T, use_pos=False), [10, 30])
```

Typical result: accuracy drops to about 0.26 and 0.18 for T = 10 and 30. Chance is 0.10.

7. Checkpoint questions

1. In Part 1, which input step has the smallest gradient? What does this mean for learning the first digit?
2. In Part 2, runs with the same settings give different accuracy. What does this tell you about training RNNs on long sequences?
3. In Part 3, accuracy drops when you remove positional encoding. Why? (Hint: what does attention know about order?)

8. Deliverable

Hand in the notebook and a one-page summary with:

- the results table (model x T) from Parts 2 and 3;
- the gradient plot and the attention plot;
- your answers to the 3 questions (2-3 sentences each).